```
from google.colab import files
uploaded = files.upload()
```

Choose Files cleaned_al…dataset.jsonl

cleaned_alpaca_dataset.jsonl(n/a) - 20568318 bytes, last modified: 11/11/2025 - 100% done

```
Saving cleaned_alpaca_dataset.jsonl to cleaned_alpaca_dataset.jsonl
```

```
# --- 🔹 1. Safe dependency setup ---
!pip install torch==2.8.0 torchvision==0.23.0 torchaudio==2.8.0 -q
!pip install -U transformers datasets peft accelerate -q

# --- 🔹 2. Imports ---
from transformers import (
    BertTokenizer,
    BertForSequenceClassification,
    Trainer,
    TrainingArguments,
    DataCollatorWithPadding
)
from datasets import load_dataset
from peft import LoraConfig, get_peft_model, TaskType
import torch
import os

# Disable WandB logging
os.environ["WANDB_DISABLED"] = "true"

# --- 🔹 3. Load dataset ---
data_path = "cleaned_alpaca_dataset.jsonl"  # Must exist from Lab 1
dataset = load_dataset("json", data_files=data_path)
dataset = dataset["train"].shuffle(seed=42).select(range(1000))

# --- 🔹 4. Tokenize text ---
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

def tokenize_fn(batch):
    combined = [i + " " + j for i, j in zip(batch["instruction"], batch["input"])]
    return tokenizer(
        combined,
        truncation=True,
        padding="max_length",   # ensures equal length
        max_length=128
    )

tokenized = dataset.map(tokenize_fn, batched=True)

# --- 🔹 5. Add scalar labels (1 or 0) ---
def make_labels(example):
    example["labels"] = 1 if len(example["output"]) > 50 else 0
    return example

tokenized = tokenized.map(make_labels)

# --- 🔹 6. Convert to PyTorch tensors ---
tokenized.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])

# --- 🔹 7. Split into train/test ---
split = tokenized.train_test_split(test_size=0.2)
train_ds, eval_ds = split["train"], split["test"]

# --- 🔹 8. Load BERT model ---
base_model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)

# --- 🔹 9. Apply LoRA adapters ---
lora_config = LoraConfig(
    task_type=TaskType.SEQ_CLS,
    r=8,
    lora_alpha=16,
    lora_dropout=0.1,
    bias="none"
)

model = get_peft_model(base_model, lora_config)
model.print_trainable_parameters()

# --- 🔹 10. Training setup ---
training_args = TrainingArguments(
    output_dir="./lora_bert_results",
    per_device_train_batch_size=8,
```

```
    per_device_eval_batch_size=8,
    num_train_epochs=1,
    save_strategy="no",
    logging_dir="./logs_lora",
    report_to="none"  # disables W&B
)

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_ds,
    eval_dataset=eval_ds,
    data_collator=data_collator,
)

# --- 🔹 11. Train and evaluate ---
trainer.train()
metrics = trainer.evaluate()
print("✅ LoRA Evaluation Metrics:", metrics)
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 511.6/511.6 kB 13.6 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 47.7/47.7 MB 19.8 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is
cudf-cu12 25.6.0 requires pyarrow<20.0.0a0,>=14.0.0; platform_machine == "x86_64", but you have pyarrow 22.0.0 which is incc
pylibcudf-cu12 25.6.0 requires pyarrow<20.0.0a0,>=14.0.0; platform_machine == "x86_64", but you have pyarrow 22.0.0 which is
Generating train split: 50795/0 [00:00<00:00, 256214.41 examples/s]
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 2.75kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 3.50MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 6.56MB/s]
config.json: 100% 570/570 [00:00<00:00, 48.1kB/s]
Map: 100% 1000/1000 [00:00<00:00, 2418.69 examples/s]
Map: 100% 1000/1000 [00:00<00:00, 12032.65 examples/s]
model.safetensors: 100% 440M/440M [00:02<00:00, 317MB/s]
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are ne
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
trainable params: 296,450 || all params: 109,780,228 || trainable%: 0.2700
[100/100 00:11, Epoch 1/1]
```

Step	Training Loss

```
[25/25 00:01]
✅ LoRA Evaluation Metrics: {'eval_loss': 0.5068771243095398, 'eval_runtime': 1.277, 'eval_samples_per_second': 156.616, 'e
```

Start coding or generate with AI.